Wafer Map Tool

Architecture, Data Flow & GUI Design

Technical Reference — v2.3

A Windows-first PyQt6 desktop application
for wafer-map visualisation and report generation

Covers: input loading (TXT/CSV, raw `.eff`), the report pipeline,
the wafer-map rendering engine (interpolate / extrapolate / "no-data" grey),
the GUI shell, and the headless automation layer.

Generated October 27, 2026

Contents

1 Overview **2**

2 System Architecture **2**

3 Data Flow **2**
3.1 Input loading — TXT / CSV 3
3.2 Input loading — raw `.eff` 3

4 Report Pipeline **3**
4.1 The two run paths (must stay in sync) 4

5 Wafer-Map Rendering Engine **4**
5.1 Grid construction (`geometry.build_wafer_grid`) 4
5.2 Interpolate vs. Extrapolate, and the “no-data” grey 4

6 Configuration Contract **5**

7 GUI Architecture **6**
7.1 Background workers & signals 6
7.2 Live preview vs. report 6

8 Automation & Scheduling **6**

9 Validation & Testing **7**

10 Recent Changes & Known Gaps **7**

1 Overview

The **Wafer Map Tool** turns semiconductor wafer measurement files into colour-coded wafer-map reports (PDF, PPTX and a CSV colour summary). It runs as a PyQt6 desktop application and, through the same codebase, as a headless command-line / scheduled-task runner.

Property	Value
Language / UI	Python 3.13, PyQt6 (Qt 6)
Numerics	NumPy, pandas, SciPy (`griddata`, `cKDTree`)
Rendering	Matplotlib (`Agg` backend — non-interactive)
Exports	PDF (vector, `pypdf` merge), CSV, PPTX (`python-pptx`)
Inputs	`.txt`/`.csv` (delimited), raw `.eff` (eSquare)
Entry points	`main.py` (GUI), `run_job.py` (headless)
Packaging	PyInstaller (`WaferMapTool.spec`); exe doubles as runner

Core idea. A measurement file is normalised to a single `pandas` table with columns `lot`, `wafer`, `x`, `y`, `value`. The pipeline groups it by (lot, wafer), interpolates each wafer onto a disc, colours it against two thresholds, and emits a multi-page report.

2 System Architecture

The code is organised by *owner / helper* boundaries: the UI layer owns interaction and preview; `report.py` owns pipeline orchestration; lower-level modules are pure/stateless where possible.

Module	Responsibility
`main.py`	Entry point. Routes `-run-job`/`-test-email`/bare `.wtjob` to the headless runner, else opens the GUI.
`wafer_tool/config.py`	`PlotConfig` (shared UI↔pipeline contract), `ColorScheme`, tuning constants.
`wafer_tool/data_loader.py`	Read/normalise delimited TXT/CSV into the 5-column DataFrame.
`wafer_tool/eff_loader.py`	Raw `.eff` reader: header scan, per-parameter streaming extraction, spec limits, value filter.
`wafer_tool/eff_pipeline.py`	GUI-free orchestration: one report per selected `.eff` parameter.
`wafer_tool/report.py`	Pipeline orchestration: load → stats → render → export.
`wafer_tool/geometry.py`	Pure maths: transforms, `build_wafer_grid`, `far_from_data_mask`.
`wafer_tool/plotting.py`	`draw_wafer_ax` — renders one wafer map (zones, grey, outline).
`wafer_tool/statistics.py`	Wafer classification and the 8-condition summary page.
`wafer_tool/exporters/`	`pdf_exporter`, `csv_exporter`, `pptx_exporter`, `report_charts`.
`wafer_tool/worker.py`	`ReportWorker` — QThread wrapper for TXT/CSV runs.
`wafer_tool/eff_worker.py`	`EffReportWorker` / `EffPreviewWorker` — QThreads for `.eff`.
`wafer_tool/ui/main_window.py`	`DataProcessorUI` — the application shell, previews, controls.
`wafer_tool/automation/`	`job.py` (`.wtjob` format), `runner.py` (headless), `scheduler*` (Windows Task Scheduler), `notify.py` (email).

3 Data Flow

At the top level, every run funnels into `generate_report()`, which is the single orchestration boundary.

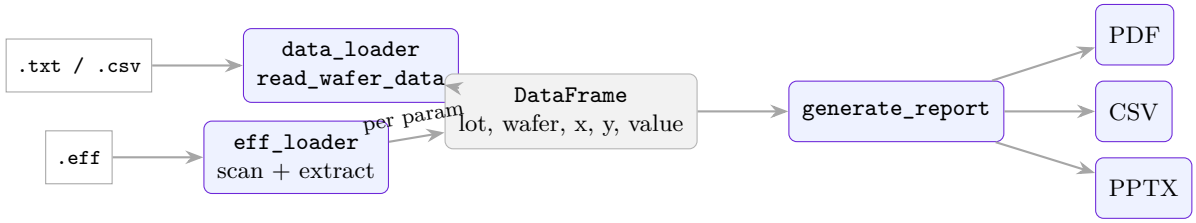


Figure 1: Top-level data flow. Both input kinds normalise to the same DataFrame; one `.eff` yields one report per selected parameter.

3.1 Input loading — TXT / CSV

`read_wafer_data` (in `data_loader.py`) parses delimited text with the fast pandas C engine, trying **tab**, then **comma**, then (last resort) a slow whitespace regex. It accepts either:

- **5+ columns**: `lot`, `wafer`, `x`, `y`, `value`; or
- **4 columns**: a combined `"LotID WaferID"` first column + `x`, `y`, `value`.

Identifier columns are forced to text (so `"01"` keeps its leading zero); `x`/`y`/`value` are parsed natively to `float32`. Rows with unparseable numbers are dropped. *Excel (`.xls`/`.xlsx`) is not currently handled by this path — see §10.*

3.2 Input loading — raw `.eff`

An `.eff` (eSquare) file is a `;`-delimited export with a metadata header, per-column `<...>` descriptor rows, then data rows. The loader exposes:

- `scan_eff(path)` — header-only scan (cheap even for GB files): detects the `lot`/`wafer`/`x`/`y` coordinate columns and the list of selectable `EffParameters`, *including each parameter's spec limits* (`<LIMIT:SPEC:LOWER_VALUE>` / `UPPER_VALUE`).
- `extract_parameters(...)` — one streaming pass writing one `lot wafer\t x\t y\t value` text file per selected parameter, into its own output folder. Supports an inclusive **value filter** (`filter_min`/`filter_max`); positions removed by the filter are remembered (`die_sites`) so the wafer outline stays full-size, and wafers emptied by the filter are recorded (`empty_wafers`).

Each extracted parameter then flows through the same `generate_report`.

4 Report Pipeline

`generate_report(filepath, config, out_dir, ...)` performs, in order:

1. **Load** data (`read_wafer_data`).
2. **Statistics**: `calculate_8_condition_statistics`.
3. **Per-wafer PNGs** (optional) — only when a live preview is attached (`on_wafer_ready`) or `render_individual_pngs=True`; headless runs skip this (it was ∼40% of runtime).
4. **PDF** (`generate_pdf`): overview charts, summary, 8-condition page, then 5×5 per-lot wafer grids. Grid pages render in parallel across processes for large reports and merge with `pypdf`.
5. **CSV** colour summary.
6. **PPTX** (if `python-pptx` present): assembled from per-page PNGs.

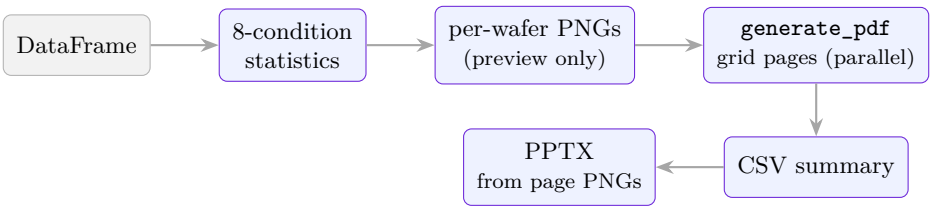


Figure 2: Report pipeline stages inside `generate_report`.

4.1 The two run paths (must stay in sync)

The *same* `generate_report` is driven two ways. Behaviour that must happen on every run (email, run-history logging) has to be wired into **both**.

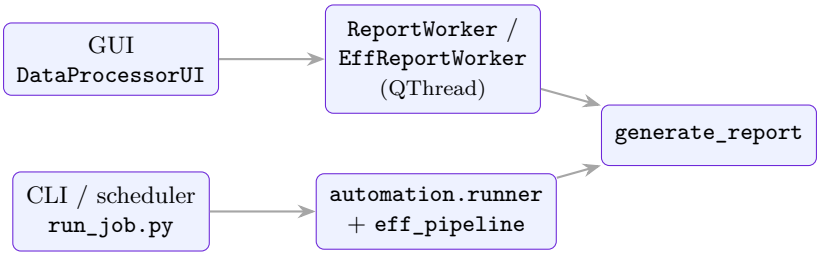


Figure 3: GUI workers and the headless runner both converge on `generate_report`.

5 Wafer-Map Rendering Engine

A single wafer is drawn by `draw_wafer_ax(ax, df, wafer_id, config, lot_sites=...)`. The geometry is built once, then two user options decide how gaps are handled.

5.1 Grid construction (`geometry.build_wafer_grid`)

1. Lay a 100×100 mesh over the wafer disc (centre + radius from the *lot* die extent, so every wafer of a lot shares one outline).
2. **Linear** `griddata` interpolates measured values between dies (NaN outside the convex hull).
3. **Nearest-neighbour** `griddata` fills the remaining NaN holes (this is the "extension"), then values are clamped to $[z_{\min}, z_{\max}]$.
4. Cells outside the wafer circle are masked to NaN.

5.2 Interpolate vs. Extrapolate, and the "no-data" grey

`far_from_data_mask` (a `cKDTree` query) flags grid cells whose nearest measured die is farther than `MAX_INTERP_DIST_CHIPS` × `DIE_PITCH` (= 4 chips). The render then branches on two `PlotConfig` flags:

Mode	`extrapolate`	Effect
Interpolate	`False`	Colour only within 4 chips of a die; farther cells are "no data".
Extrapolate	`True`	Keep the nearest-fill everywhere — the whole disc is coloured.

	`show_missing_grey`	Effect on "no-data" cells (Interpolate only)
Grey on	`True`	Painted grey (`COLOR_MISSING`); "No data" legend entry.
Grey off	`False`	Left blank/white (wafer outline still drawn).

Listing 1: The interpolate/extrapolate + grey decision in draw_wafer_ax.

```
if config.extrapolate:
    missing = None # keep the full nearest-fill
else:
    missing = (far_from_data_mask(xi, yi, x, y,
                                  MAX_INTERP_DIST_CHIPS * DIE_PITCH)
               & ~np.isnan(zi)) # cells > 4 chips from any die

if missing is not None:
    zi[missing] = np.nan # drop "no data" cells from the fill
```

```
# ... contourf the three threshold zones (low / mid / high) ...
if missing is not None and config.show_missing_grey:
    ax.contourf(xi, yi, np.where(missing, 1.0, np.nan),
                levels=[0.5, 1.5], colors=[COLOR_MISSING]) # grey overlay
# wafer outline (neutral grey, visible on any background)
ax.add_patch(Circle((x_mid, y_mid), radius, fill=False,
                    color="#888888", linewidth=1.0))
```

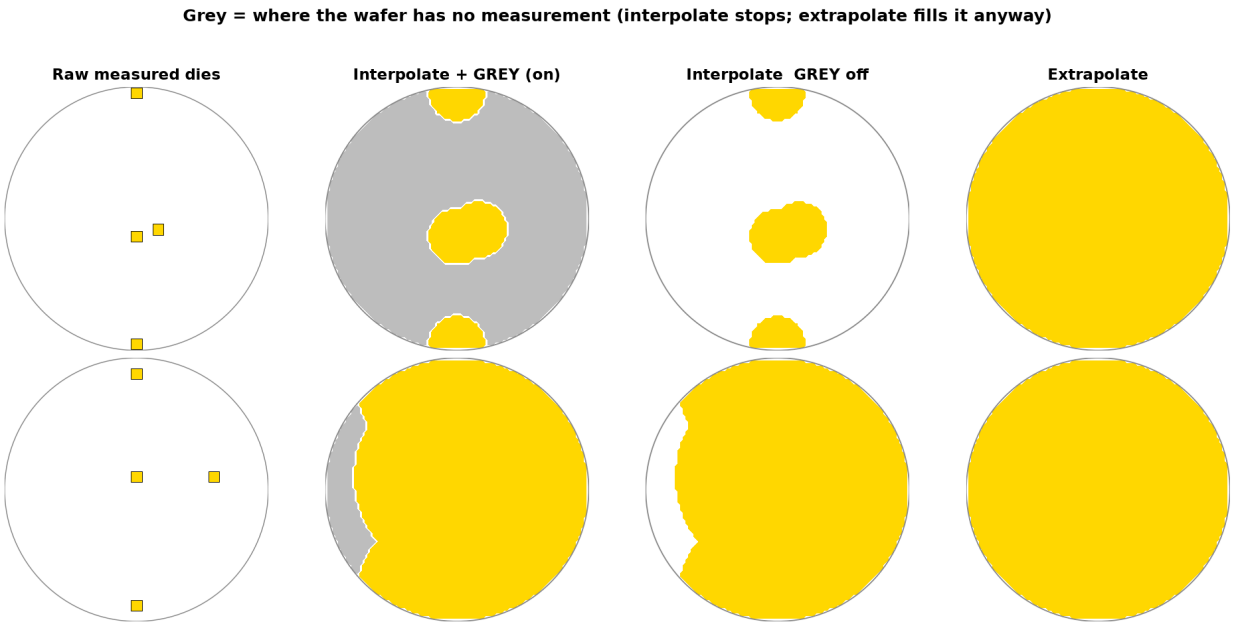


Figure 4: Two sparsely-sampled wafers (4 of 10 sites). **Grey = where there is no measurement.** Left→right: raw dies; interpolate with grey on; same with grey off (blank); extrapolate (whole wafer filled from the nearest readings).

Colouring. Values map to three zones against the two thresholds (t_{low}, t_{high}) via a `contourf` with the run's `ColorScheme` (*Reverse / High=Green* swaps low↔high colours). `use_log` applies a $\log_{10}$ mapping (magnitudes clamped to avoid $\log 0$). Spatial `mirror_x/mirror_y/rot_deg` transforms are applied before gridding. *All* 256 *combinations of these options were exercised without error (§9).*

6 Configuration Contract

`PlotConfig` is the single typed object passed from UI state into the pipeline and the renderer.

Field	Type	Meaning
`t_low`, `t_high`	float	Lower/upper thresholds (normalised low≤high).
`use_log`	bool	$\log_{10}$ value mapping.
`high_is_green`	bool	Reverse the zone colours (high = green).
`mirror_x`, `mirror_y`	bool	Spatial mirror.
`rot_deg`	int	Clockwise rotation $\in \{0, 90, 180, 270\}$.
`show_missing_grey`	bool	**(new)** Grey the "no-data" areas vs. blank.
`extrapolate`	bool	**(new)** Fill whole wafer vs. interpolate-near-dies.

Key tuning constants (`config.py`): `GRID_RESOLUTION=100`, `DIE_PITCH=1.0`, `MAX_INTERP_DIST_CHIPS=4`, `MIN_POINTS_FOR_PLOT=4`, `MAX_WAFERS_PER_PAGE=25`, `PAGE_PNG_DPI=110`, `WAFER_PNG_DPI_UNIFORM=70`, `WAFER_PNG_DPI_MULTIZONE=150`, `COLOR_MISSING` (grey, #BDBDBD).

7 GUI Architecture

`DataProcessorUI` (a `QMainWindow`) splits into a **left control panel** (now inside a `QScrollArea`, so controls never overlap on short windows) and a **right preview panel** with tabs.

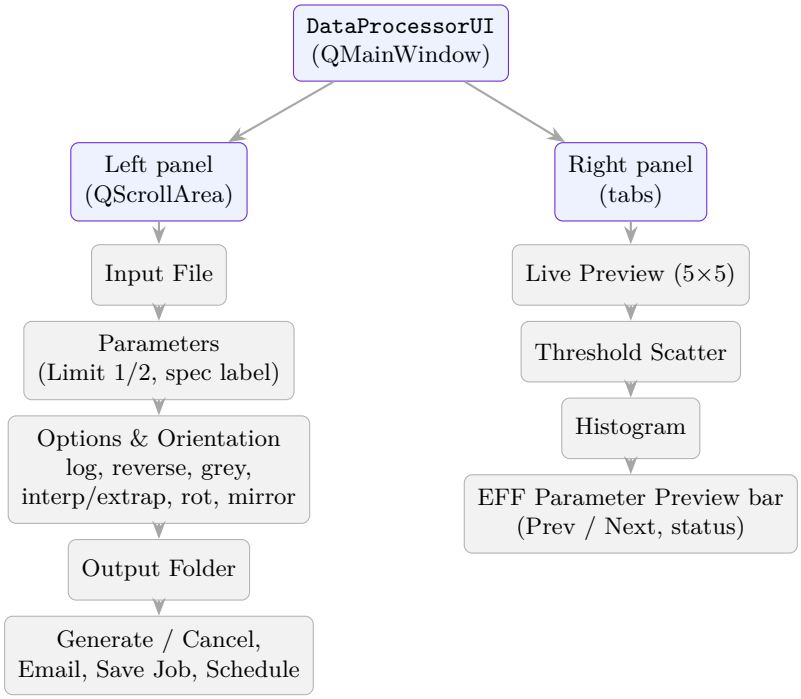


Figure 5: GUI widget tree. The *Options & Orientation* group hosts the new grey / interpolate / extrapolate controls.

7.1 Background workers & signals

Heavy work never runs on the UI thread. Three `QThread`s:

- `ReportWorker` — TXT/CSV report. Signals: `progress`, `wafer_progress`, `status_message`, `wafer_ready`, `report_done`, `error`.
- `EffReportWorker` — one report per selected `.eff` parameter (wraps `eff_pipeline.process_eff`). Adds `param_done` / `all_done`.
- `EffPreviewWorker` — reads one parameter in the background to feed the scatter/histogram preview without blocking the UI.

Cancellation is cooperative (a `threading.Event` checked between wafers). The **wafer-map options** (§5) are read from the Options group into a `PlotConfig` at launch (both the TXT and `.eff` run handlers), and persisted into saved `.wtjob`s via `_PLOT_KEYS`.

7.2 Live preview vs. report

The *Threshold Scatter* and *Histogram* tabs are value-distribution views (no wafer geometry); the grey / interpolate / extrapolate options do *not* affect them. Those options change the *wafer maps* — the *Live Preview (5×5)* grid (filled as wafers render) and the PDF/PPTX.

8 Automation & Scheduling

A run can be saved as a `.wtjob` (JSON) capturing the input, output folder and every `PlotConfig` field (now including `show_missing_grey` and `extrapolate`), plus `.eff` parameter selection, the value filter, and email settings.

- `run_job.py <job.wtjob>` replays a job headlessly via `automation.runner.main`.
- `scheduler.py` / `scheduler_dialog.py` register jobs with Windows Task Scheduler; the frozen `WaferMapTool.exe` is invoked as `WaferMapTool.exe -run-job <job>`.
- `notify.py` optionally emails links (PDF / PPTX / output folder) via Outlook COM or SMTP (credentials from environment variables).
- Every run appends a row to `run_history.csv`; the general log is `wafer_tool.log`. Both writes are non-fatal.

9 Validation & Testing

The current build was exercised as follows.

Check	Result
Full pipeline on **all 6** `.eff` files, both modes (real `generate_report`: PDF+CSV+PPTX)	**PASS** — 6/6 interpolate, 6/6 extrapolate; valid multi-page PDFs.
Render-option combinations (`extrapolate`×`grey`×`log`×`reverse`×`mirror`×`rotation`)	**768/768 OK**, 0 crashes (across dense, sparse and all-zero wafers).
Automation unit tests (`test_automation.py`)	**18/18 passed**.

10 Recent Changes & Known Gaps

Added in this cycle:

- "*Show missing data in grey*" checkbox and an *Interpolate / Extrapolate* choice (new `PlotConfig` fields, honoured by the renderer and both run paths, persisted in `.wtjobs`).
- Wafer outline drawn in neutral grey so it is visible on any background (previously white ⇒ invisible on the white report background).
- Left control panel wrapped in a scroll area (fixes Rotation/Mirror crowding on short windows).

Known gaps (candidates for the next cycle):

- **Excel import**: `.xls`/`.xlsx` are *not* read by `data_loader` (CSV/TXT only). Adding `pd.read_excel` + a file-dialog filter would close this.
- **Value filter for TXT/CSV**: the inclusive value filter exists only on the `.eff` path; TXT/CSV imports are unfiltered.